CO1-ASSESMENT TOOL -2

CSA4707- (Deep Learning)

A.Sai Joshitha (192424220)

1. MACHINE LEARNING BASICS

1. Aim

To predict the likelihood of diabetes using patient data by implementing Linear Regression and Logistic Regression and comparing their performance.

2. Dataset Description

The Pima Indians Diabetes dataset contains 768 patient records and 9 columns, including 8 input features and the target variable Outcome. Outcome 0 represents no diabetes, and Outcome 1 represents diabetes.

```
Dataset Shape: (768, 9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outc
0	6	148	72	35	0	33.6	0.627	50	
1	1	85	66	29	0	26.6	0.351	31	
2	8	183	64	0	0	23.3	0.672	32	
3	1	89	66	23	94	28.1	0.167	21	
4	0	137	40	35	168	43.1	2.288	33	

3. Data Preprocessing

The dataset was inspected for missing values, duplicates, and invalid zero measurements. Appropriate zero measurements were treated as missing values and handled using median imputation. The dataset was divided into training and testing sets, and feature scaling was applied where appropriate.

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33

```
Training set class distribution:
Outcome
0    0.651466
1    0.348534
Name: proportion, dtype: float64

Testing set class distribution:
Outcome
0    0.649351
1    0.350649
Name: proportion, dtype: float64
```

4. Algorithms Used

Linear Regression: Predicts continuous numerical values. Its predictions were thresholded to obtain binary class labels for comparison.

Logistic Regression: Predicts the probability of a patient belonging to the diabetes class. It is more suitable for binary classification.

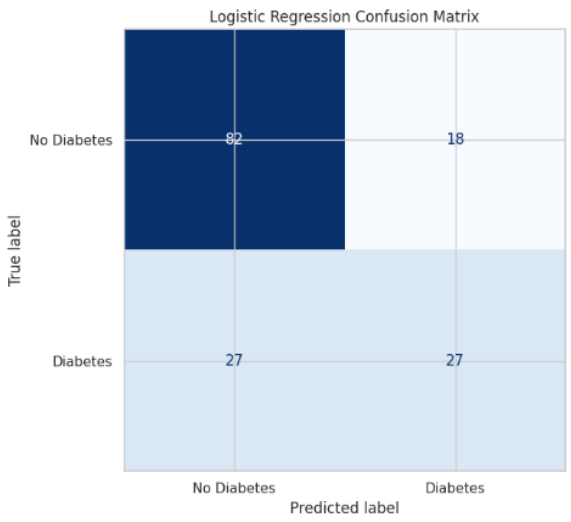
	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7078	0.6000	0.5000	0.5455	0.8130
Linear Regression (Thresholded)	0.7013	0.5909	0.4815	0.5306	0.8116

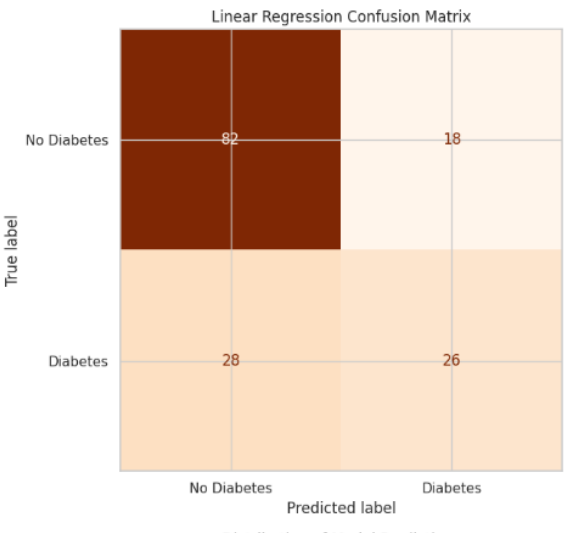
5. Implementation

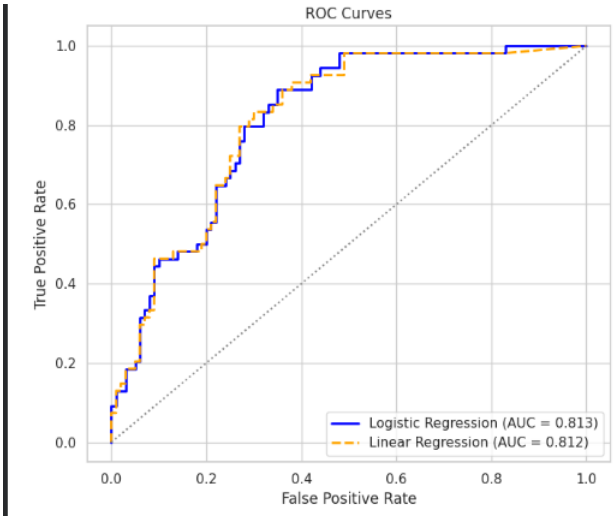
The models were implemented in Python using Pandas, NumPy, Matplotlib and Scikit-learn in Google Colab.

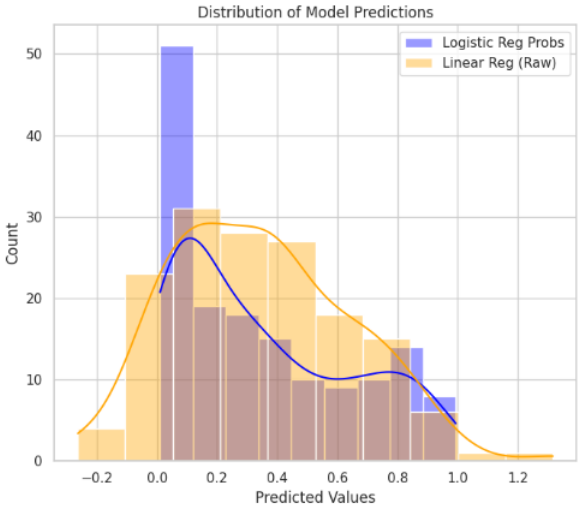
6. Results and Performance Evaluation

The models were evaluated using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices. MAE, RMSE and R-squared were also calculated for Linear Regression.









7. Result

Both models were trained and evaluated on the diabetes dataset. Their actual performance metrics were compared to identify the more suitable approach for diabetes classification.

8. Conclusion

This experiment demonstrated dataset preprocessing, machine learning model training and performance evaluation. Logistic Regression is more suitable for binary diabetes classification, while Linear Regression serves as a baseline for comparison. The results also highlight the importance of evaluating recall and other metrics in addition to accuracy.

2. LEARNING ALGORITHMS

1. Aim

To implement different learning algorithms for recommending products to users based on their previous purchases or ratings and compare their performance.

2. Dataset Description

A product recommendation dataset containing user IDs, product IDs, and purchase history or ratings is used. The dataset helps identify user preferences and recommend relevant products.

Dataset generated with 902 unique interactions.

	UserID	ProductID	Rating
0	52	34	1
1	93	47	3
2	15	8	2
3	72	40	4
4	61	49	4

3. Data Preprocessing

The dataset was inspected for missing values and duplicate records. Relevant user, product, and rating information was selected and prepared for model training. The data was divided into training and testing sets to evaluate model performance.

4. Algorithms Used

K-Nearest Neighbors (KNN): Recommends products based on similarities between users, products, or their interaction patterns.

```
NearestNeighbors
NearestNeighbors(algorithm='brute', metric='cosine')
```

Decision Tree: Learns patterns from user or product features to predict ratings or purchase preferences.

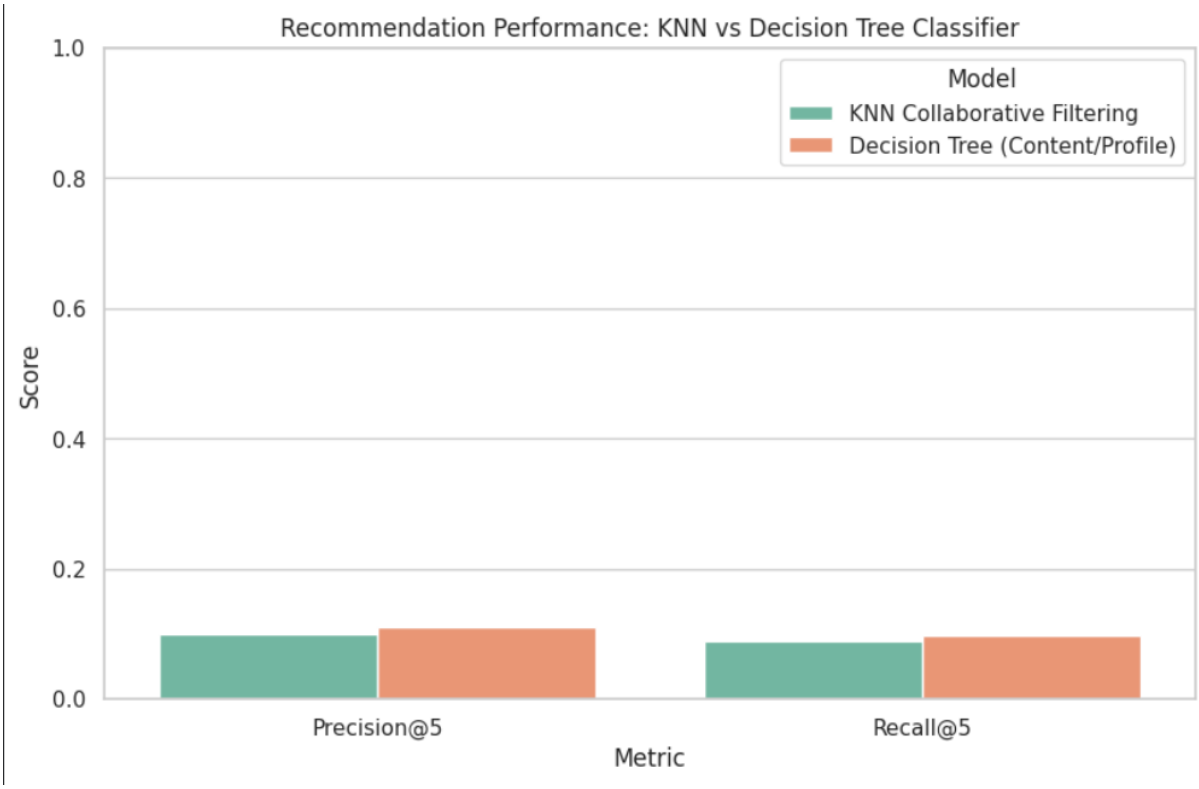
```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=5, random_state=42)
```

5. Implementation

The algorithms were implemented in Python using Pandas, NumPy, and Scikit-learn in Google Colab. Hyperparameters were adjusted to improve model performance.

6. Results and Performance Evaluation

The models were evaluated using appropriate metrics, such as Precision@K and Recall@K for recommendations, or MAE and RMSE when predicting ratings. The results were compared to identify the better-performing approach.



7. Result

The KNN and Decision Tree algorithms were implemented and evaluated for the product recommendation task. Their performance was compared using the calculated evaluation metrics.

8. Conclusion

This case study demonstrated how learning algorithms can be applied to product recommendation. Hyperparameter tuning helps improve model performance, while the choice of algorithm depends on the dataset, recommendation method, and evaluation result.

3. CAPACITY, OVERFITTING AND UNDERFITTING

1. Aim

To develop house price prediction models with different complexities and analyze underfitting, overfitting, and the effect of model complexity on prediction performance.

2. Dataset Description

A house price dataset is used, containing features such as location, average rooms, population, and other housing-related information. The target variable represents the house value to be predicted.

```
Features shape: (20640, 8)
Target shape: (20640,)
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

3. Data Preprocessing

The dataset was inspected for missing values and duplicate records. Input features and the target variable were separated, and the data was divided into training and testing sets. Feature preprocessing was performed using a suitable pipeline to avoid data leakage.

```
Training set: (16512, 8)
Testing set: (4128, 8)
```

4. Algorithms Used

Linear Regression: Models a linear relationship between input features and house prices.

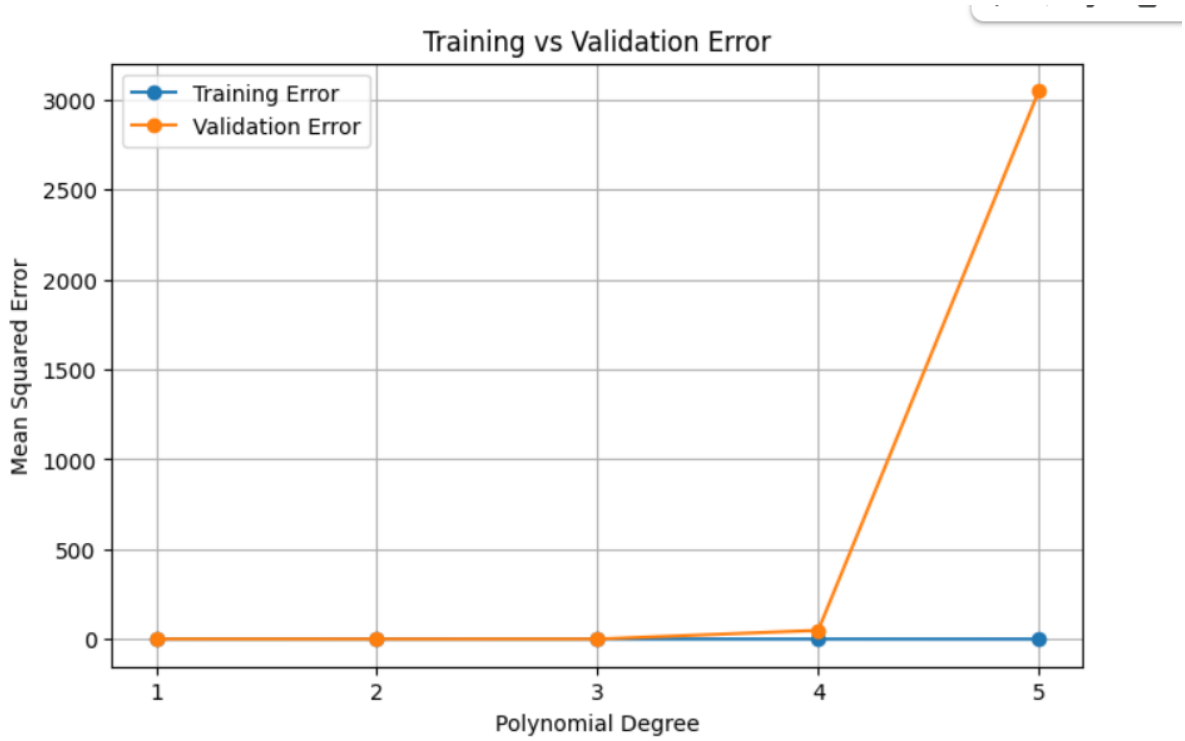
Polynomial Regression: Adds polynomial features to represent more complex relationships between input features and house prices. Different polynomial degrees were tested to study model complexity.

5. Implementation

The models were implemented in Python using Pandas, NumPy, Matplotlib, and Scikit-learn in Google Colab. Polynomial degrees were compared, and cross-validation was used on the training data to select a suitable model.

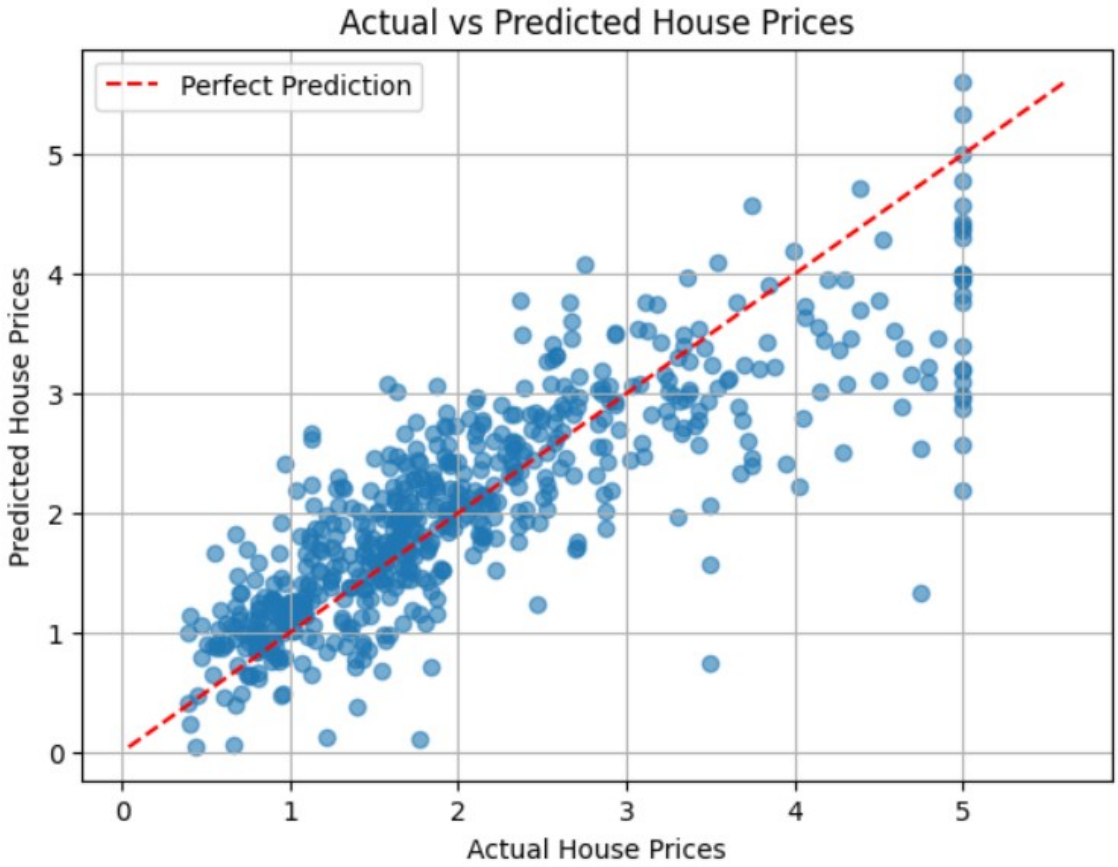
6. Results and Performance Evaluation

The models were evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared. Training and validation errors were compared to understand the effect of model complexity.



7. Analysis

Underfitting occurs when a model is too simple to capture important patterns in the data. Overfitting occurs when a model fits the training data very well but performs poorly on unseen data. A suitable model complexity balances prediction accuracy and generalization.



8. Result

Linear Regression and Polynomial Regression models were implemented and evaluated. Their performance was compared to understand how model complexity affects house price prediction.

9. Conclusion

This case study demonstrated the importance of selecting an appropriate model complexity. Cross-validation and training/testing error comparisons help identify models that generalize better and reduce the risk of underfitting and overfitting.

4.ESTIMATION, BIAS AND VARIANCE

1. Aim

To study estimation, bias, and variance in machine learning by developing prediction models and comparing their performance on training and testing data.

2. Dataset Description

A suitable dataset is the California Housing dataset, which contains housing-related features and house prices. The dataset is used to train and evaluate regression models.

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	1.6812	25.0	4.192201	1.022284	1392.0	3.877437	36.06	-119.01	0.47700
1	2.5313	30.0	5.039384	1.193493	1565.0	2.679795	35.14	-119.46	0.45800
2	3.4801	52.0	3.977155	1.185877	1310.0	1.360332	37.80	-122.44	5.00001
3	5.7376	17.0	6.163636	1.020202	1705.0	3.444444	34.28	-118.72	2.18600
4	3.7250	34.0	5.492991	1.028037	1063.0	2.483645	36.62	-121.93	2.78000

3. Data Preprocessing

The dataset is checked for missing values and duplicate records. Input features and the target variable are separated, and the dataset is divided into training and testing sets. Feature scaling is applied where necessary.

```
Training set shape: (2400, 8)
Testing set shape: (600, 8)
```

4. Algorithms Used

- Linear Regression
- Polynomial Regression
- Bias and Variance Analysis

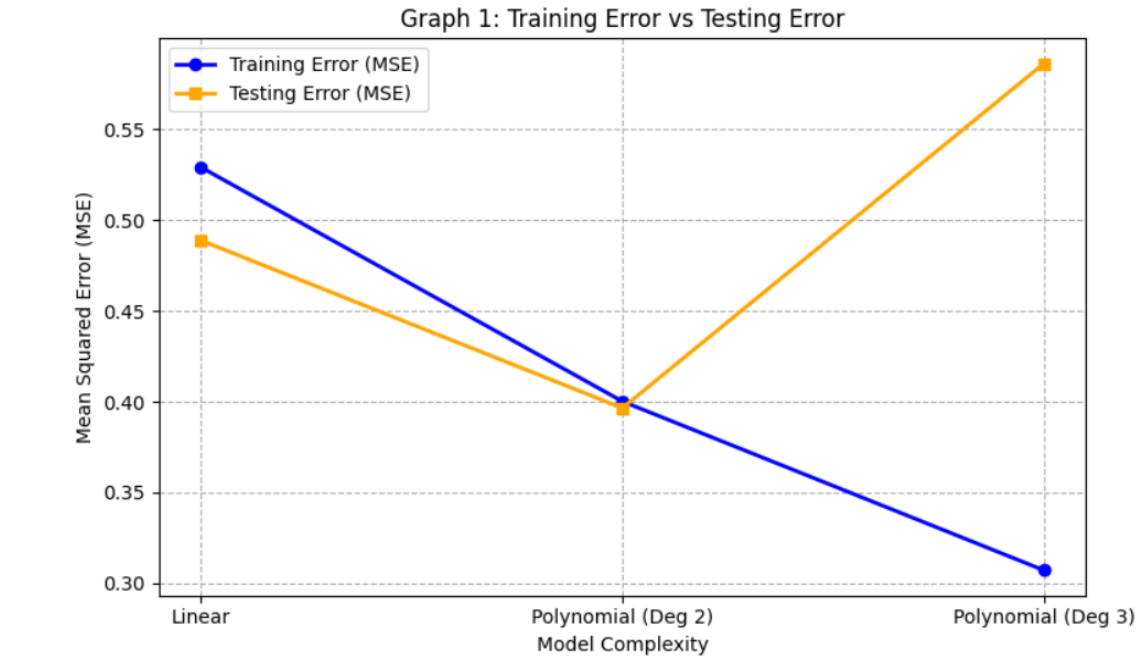
	Model	Train MAE	Test MAE	Train MSE	Test MSE	Train RMSE	Test RMSE	Train R^2	Test R^2
0	Linear Regression	0.533007	0.524499	0.528984	0.488734	0.727313	0.699095	0.595400	0.625249
1	Polynomial Degree 2	0.456487	0.463098	0.400162	0.396476	0.632584	0.629663	0.693931	0.695990
2	Polynomial Degree 3	0.404843	0.462724	0.307218	0.586035	0.554273	0.765529	0.765020	0.550640

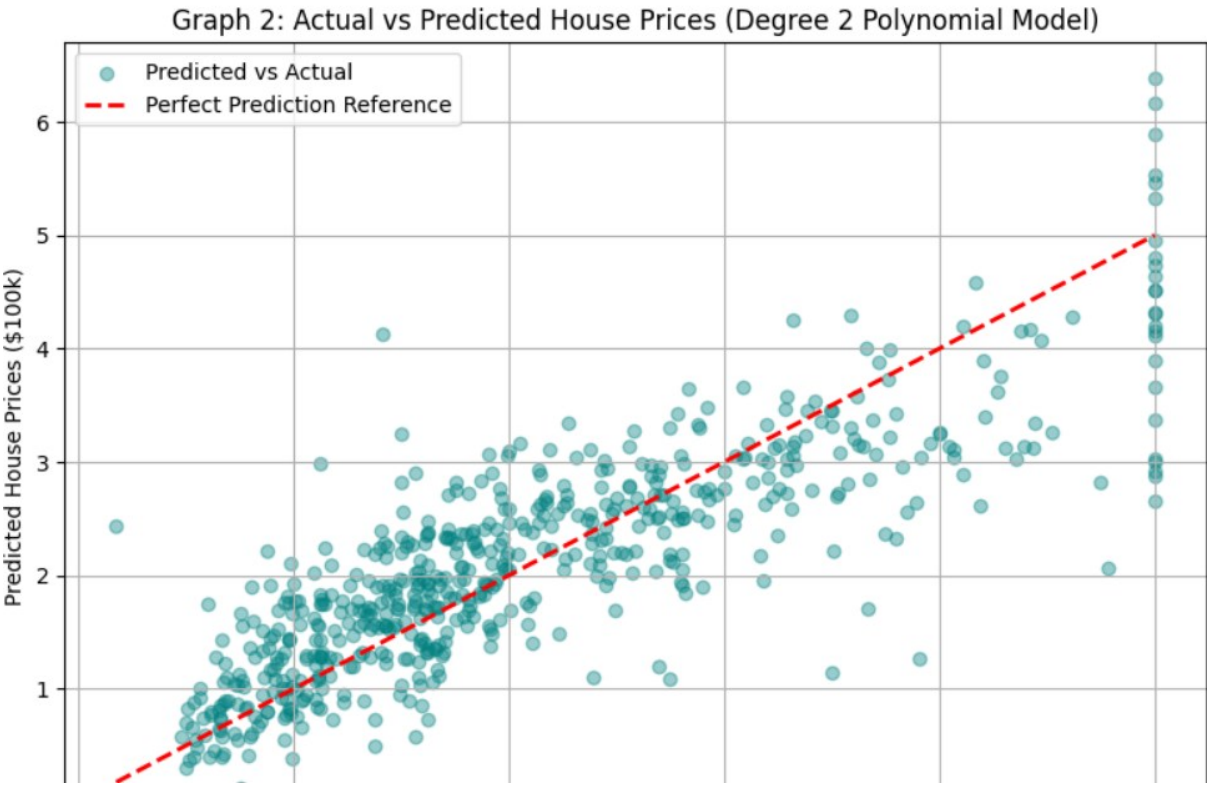
5. Implementation

The models are implemented using Python, Pandas, NumPy, Matplotlib, and Scikit-learn in Google Colab. Predictions are generated and compared with the actual house prices.

6. Results and Evaluation

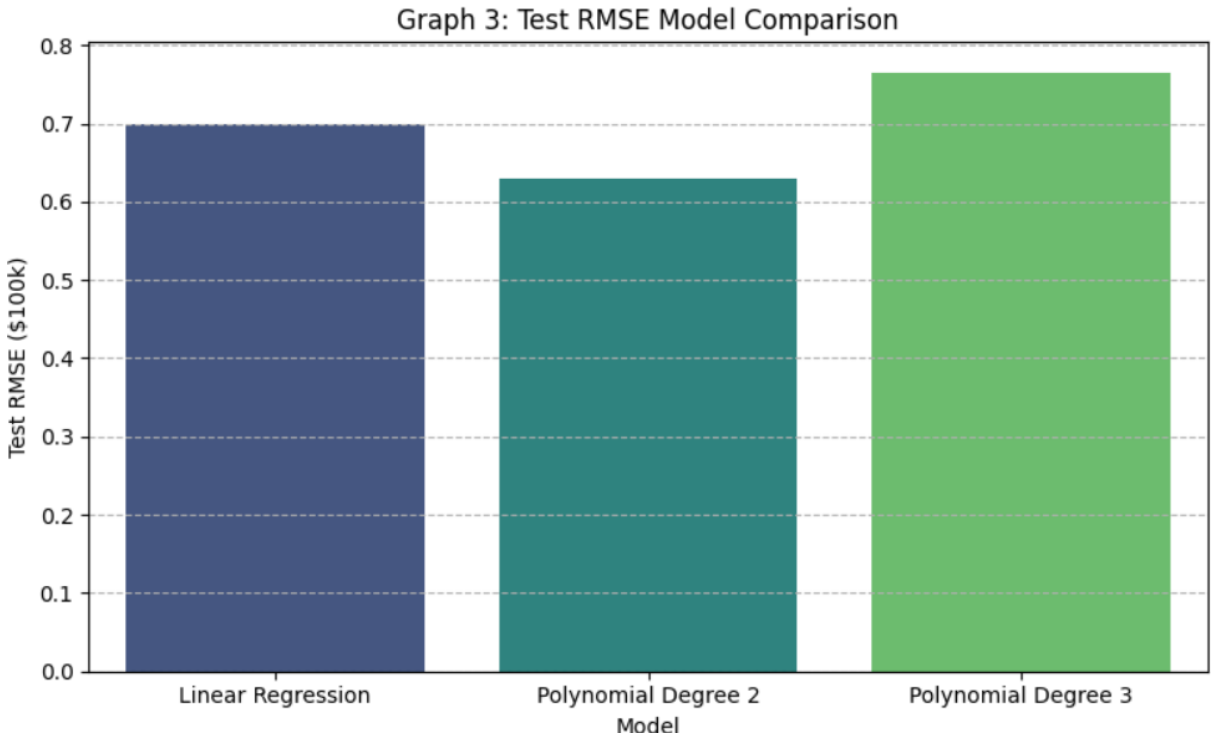
The models are evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2). Training and testing errors are compared to understand model performance.





7. Analysis

Bias represents errors caused by overly simple assumptions in a model. Variance represents how much the model's predictions change with variations in the training data. A model with high bias may underfit, while a model with high variance may overfit. The goal is to achieve a balance that provides good performance on unseen data.



8. Result

The regression models were trained and evaluated using the selected dataset. Their error metrics and graphs were used to analyze bias, variance, and generalization performance.

9. Conclusion

This case study demonstrates the importance of balancing bias and variance when building machine learning models. A suitable model should learn meaningful patterns without overfitting the training data.

5. BAYESIAN STATISTICS

1. Aim

To apply Bayesian statistics to estimate probabilities and update predictions using prior knowledge and observed data.

2. Dataset Description

A suitable classification dataset is the Breast Cancer Wisconsin dataset available in Scikit-learn. It contains numerical features extracted from breast cell samples and a target indicating benign or malignant classification.

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67

5 rows × 31 columns

3. Data Preprocessing

The dataset is checked for missing values and duplicate records. Input features and target labels are separated, and the data is divided into training and testing sets. Feature scaling is applied where required.

```
Training instances: 455 (62.64% Benign)
Testing instances: 114 (63.16% Benign)
```

4. Algorithms Used

- Gaussian Naive Bayes
- Bayesian probability estimation
- Confusion matrix and classification metrics

```
=== Test Set Evaluation Metrics ===
Accuracy : 0.9298
Precision: 0.9444
Recall   : 0.9444
F1 Score : 0.9444

Detailed Classification Report:
              precision    recall  f1-score   support

   malignant       0.90      0.90      0.90        42
      benign       0.94      0.94      0.94        72

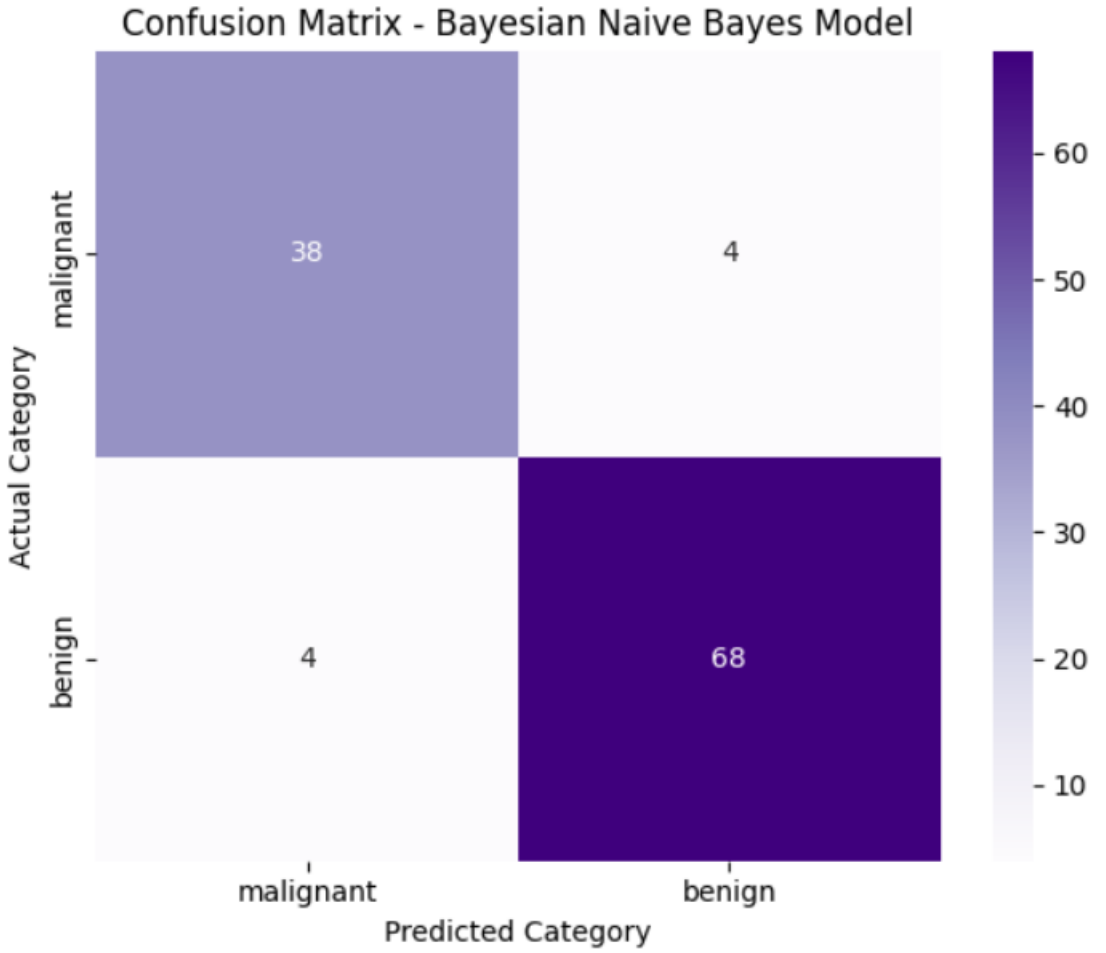
    accuracy                           0.93       114
   macro avg       0.92      0.92      0.92       114
weighted avg       0.93      0.93      0.93       114
```

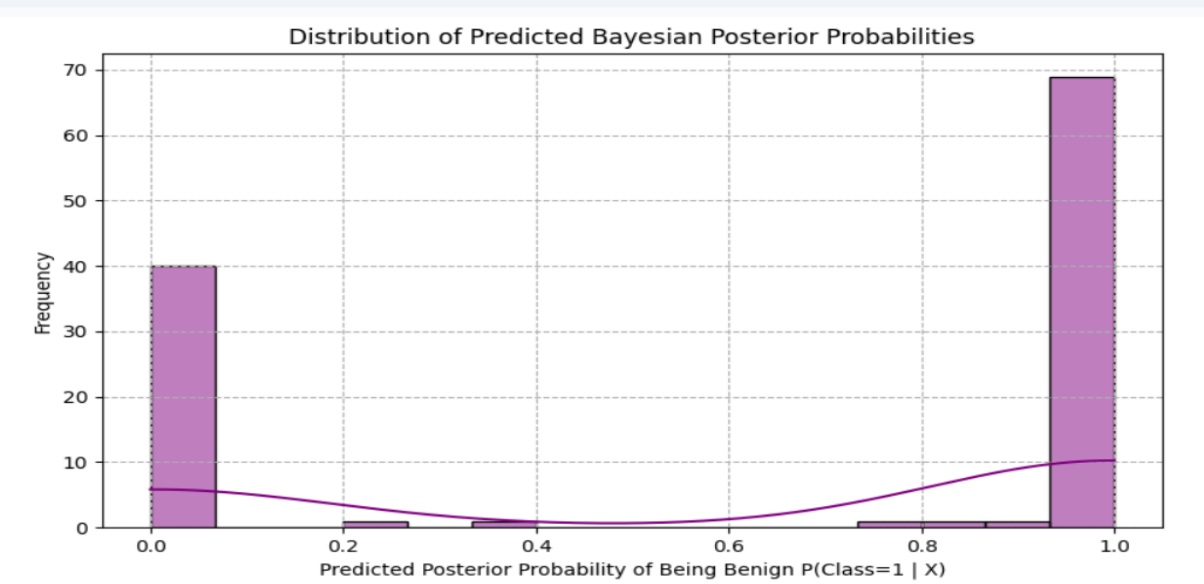
5.Implementation

The implementation is carried out in Python using Pandas, NumPy, Matplotlib, and Scikit-learn in Google Colab. The Gaussian Naive Bayes model is trained to predict the target class using the input features.

6. Results and Evaluation

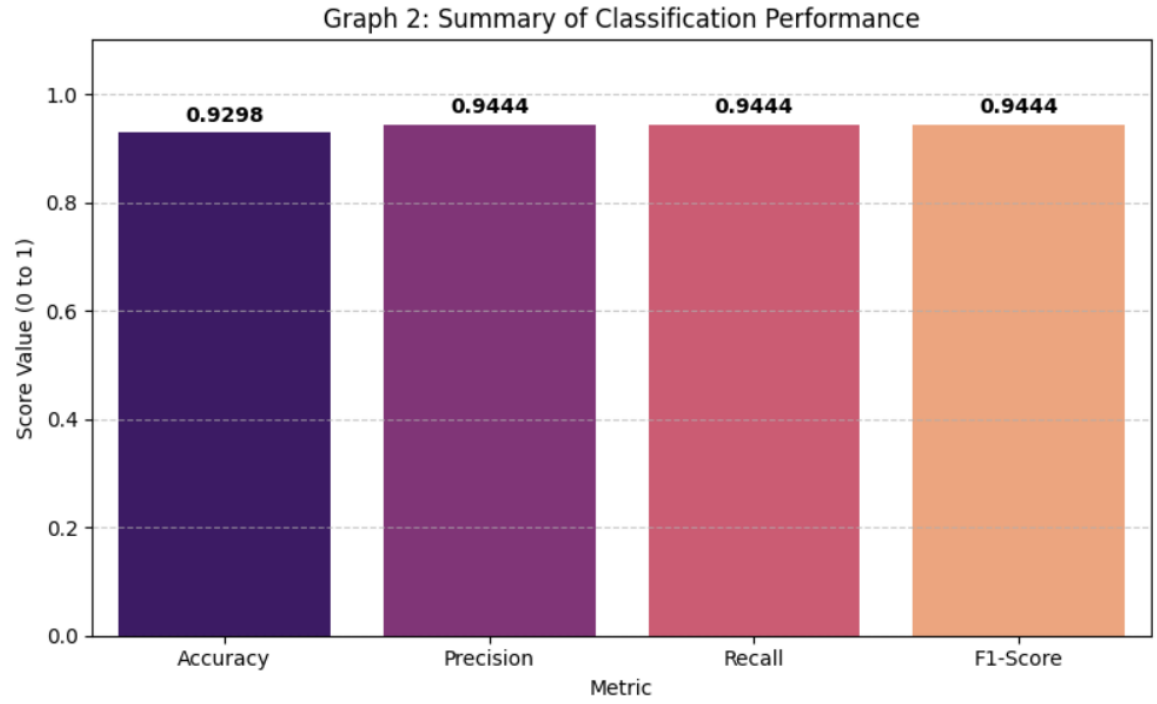
The model is evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. The actual results are used to understand the model's classification performance.

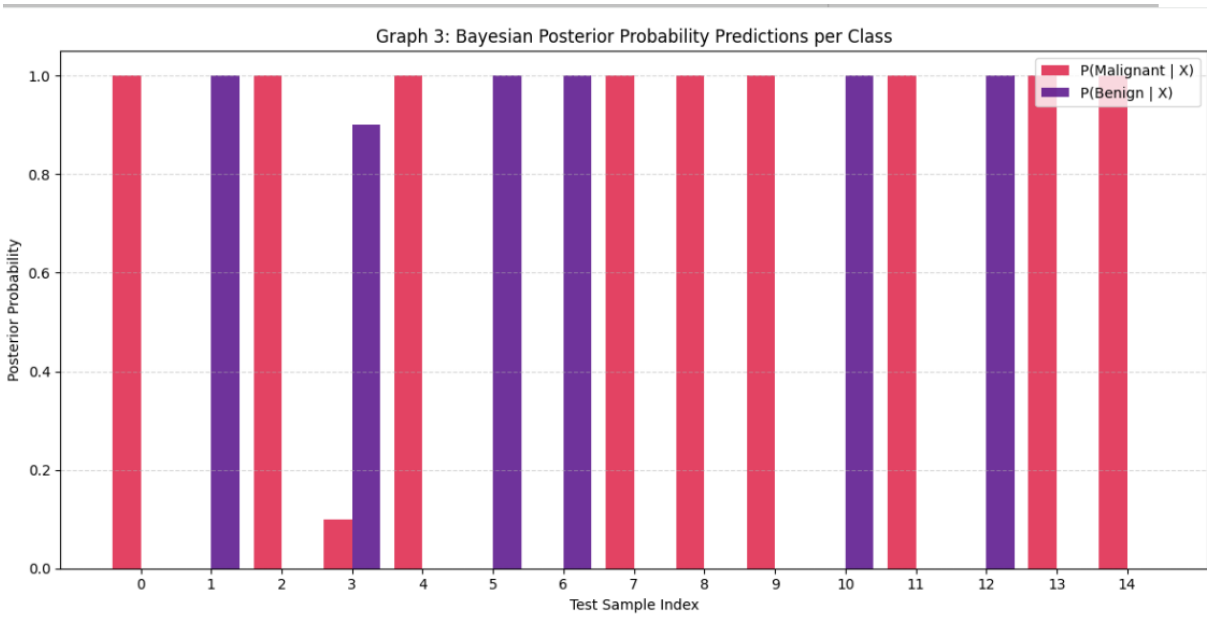




7. Analysis

Bayesian statistics updates probability estimates using observed evidence. Naive Bayes applies Bayes' theorem while assuming conditional independence between features given the class. The evaluation results show how effectively the model classifies the samples.





8. Result

The Gaussian Naive Bayes model was trained and evaluated on the selected dataset. Its performance was measured using classification metrics and a confusion matrix.

9. Conclusion

This case study demonstrates the application of Bayesian statistics to classification. Bayes' theorem provides a way to update probabilities based on evidence, while Gaussian Naive Bayes offers a simple probabilistic classification method.

6. STOCHASTIC GRADIENT DESCENT

1. Aim

To implement Stochastic Gradient Descent for training a machine learning model and analyze how iterative parameter updates minimize prediction error.

2. Dataset Description

The California Housing dataset is used to predict house prices based on housing-related features. It contains numerical input features and a continuous target variable representing house values.

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	1.6812	25.0	4.192201	1.022284	1392.0	3.877437	36.06	-119.01	0.47700
1	2.5313	30.0	5.039384	1.193493	1565.0	2.679795	35.14	-119.46	0.45800
2	3.4801	52.0	3.977155	1.185877	1310.0	1.360332	37.80	-122.44	5.00001
3	5.7376	17.0	6.163636	1.020202	1705.0	3.444444	34.28	-118.72	2.18600
4	3.7250	34.0	5.492991	1.028037	1063.0	2.483645	36.62	-121.93	2.78000

3. Data Preprocessing

The dataset is checked for missing values and duplicate records. Input features and the target variable are separated. The dataset is divided into training and testing sets, and feature scaling is applied where required.

```
Training Set: 4000 samples
Testing Set : 1000 samples
```

4. Algorithm Used

- Stochastic Gradient Descent Regressor (SGDRegressor)
- Feature scaling
- Mean Squared Error (MSE) and R-squared (R^2) evaluation

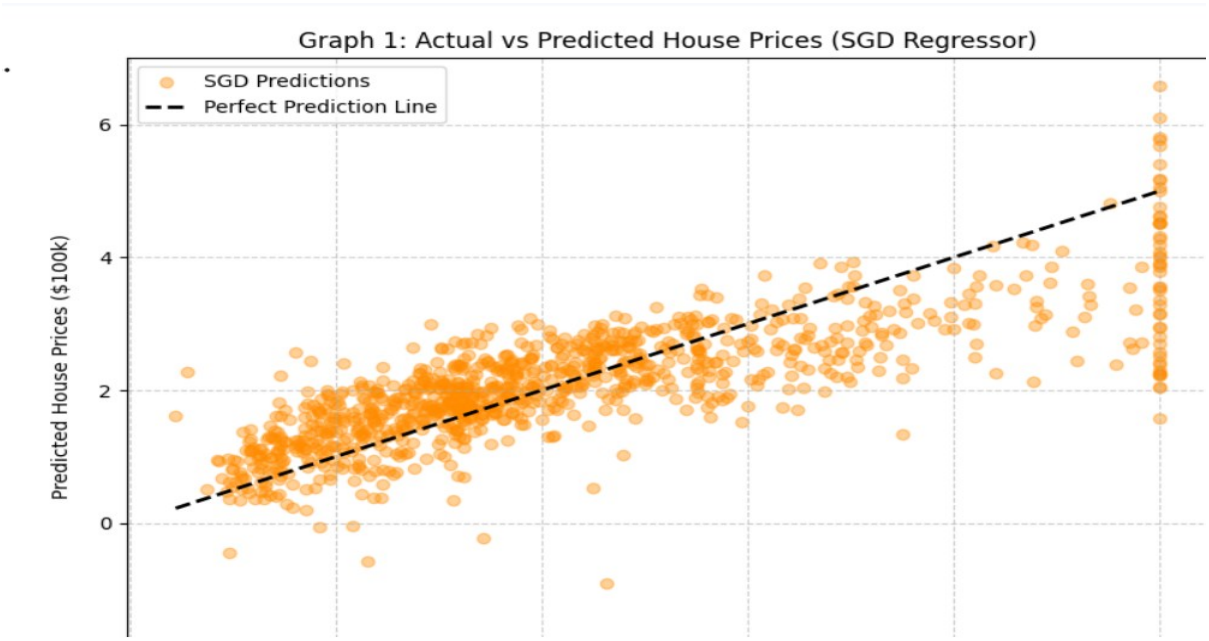
	Metric	Linear Regression (OLS)	SGD Regressor
0	MAE	0.522092	0.527149
1	MSE	0.501215	0.517863
2	RMSE	0.707965	0.719627
3	R^2 Score	0.628906	0.616580

5. Implementation

The implementation is performed in Python using Pandas, NumPy, Matplotlib, and Scikit-learn in Google Colab. The SGD model learns by updating its parameters using gradients calculated from individual training samples or small batches.

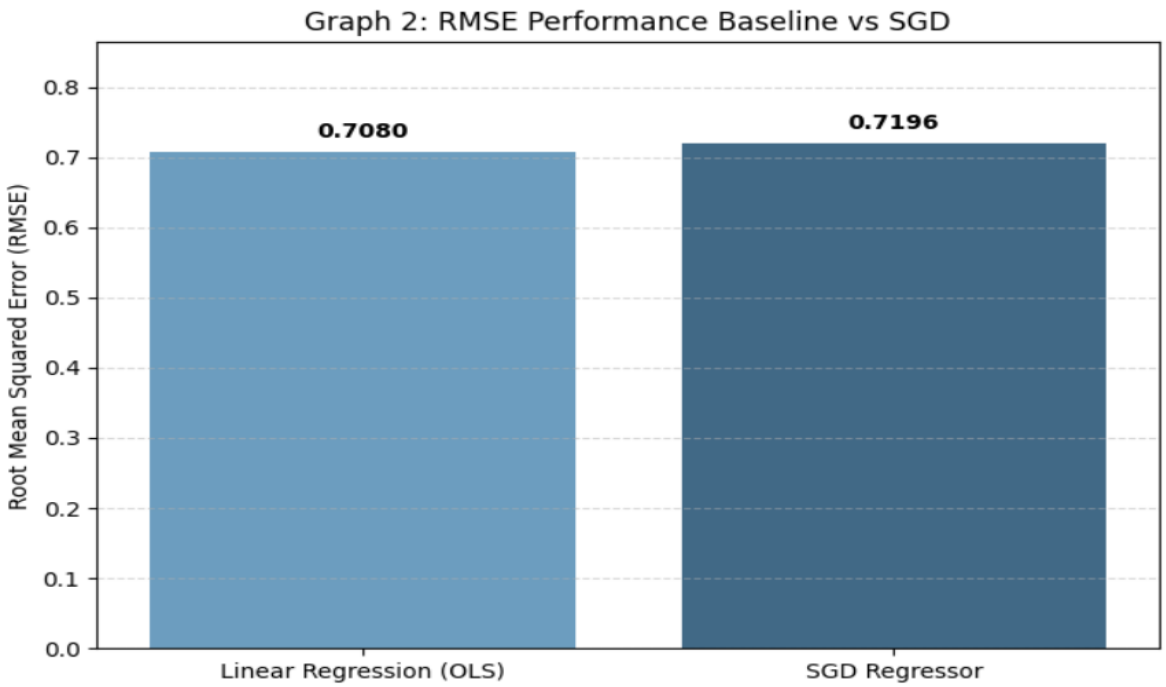
6. Results and Evaluation

The trained model is evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2). The results are used to assess prediction performance.



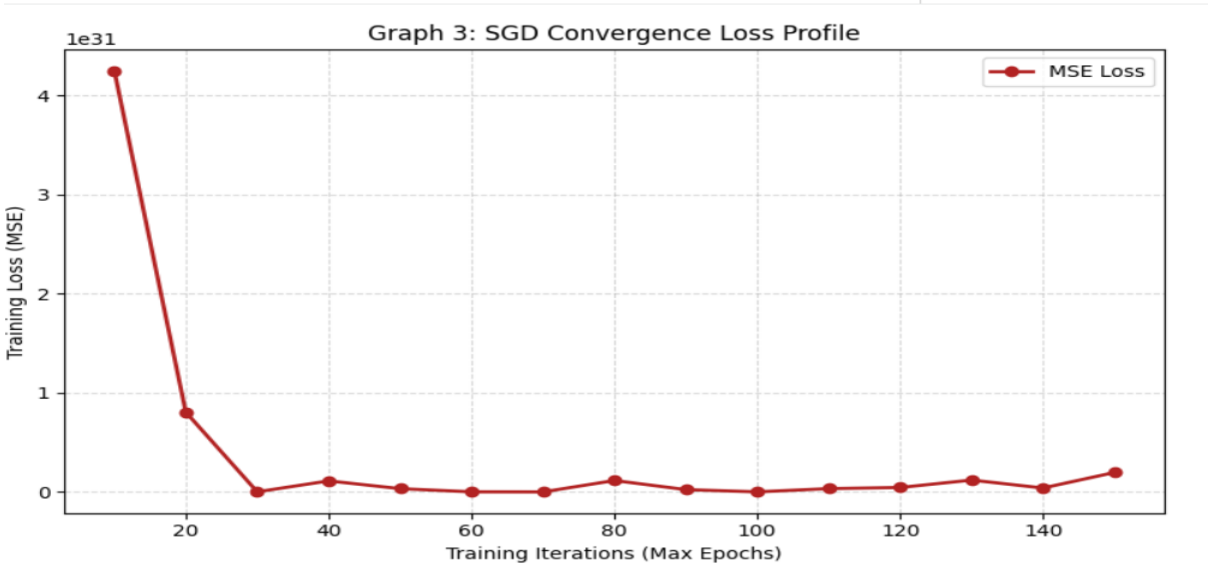
7. Analysis

Stochastic Gradient Descent updates model parameters iteratively to reduce the loss function. The learning rate controls the size of each update. A learning rate that is too high may cause unstable learning, while a very low learning rate may slow convergence.



8. Result

The SGD regression model was trained on the housing dataset and evaluated using regression metrics. Its performance was analyzed using the calculated errors and prediction results.



9. Conclusion

This case study demonstrates how Stochastic Gradient Descent can optimize a regression model through iterative parameter updates. Proper feature scaling and learning-rate selection are important for stable training.

7.CHALLENGES MOTIVATING DEEP LEARNING

1. Aim

To understand the major challenges in traditional machine learning and demonstrate how deep learning addresses these challenges using handwritten digit classification.

2. Introduction

Deep learning is a branch of machine learning that uses artificial neural networks with multiple layers to learn complex patterns from data. Traditional machine learning often requires manual feature extraction, whereas deep learning can automatically learn useful features from large datasets. It is widely used in image recognition, speech processing, medical diagnosis, and natural language processing.

3. Challenges Motivating Deep Learning

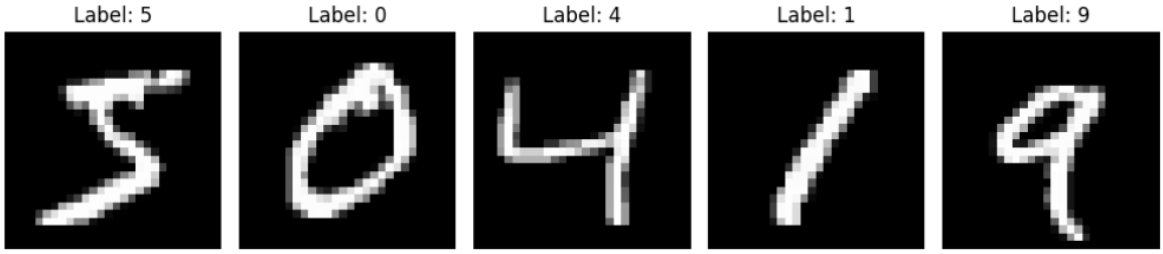
The major challenges that motivate the use of deep learning are:

- **Feature Engineering:** Manually identifying useful features can be difficult and time-consuming.
- **Complex Data:** Images and other unstructured data contain complex patterns.
- **Nonlinear Relationships:** Traditional models may struggle to learn complicated relationships.
- **Large Datasets:** Processing large datasets requires efficient computational resources.
- **Overfitting:** A model may perform well on training data but poorly on unseen data.
- **Generalization:** A model should make accurate predictions on new data.

4. Dataset Description

The MNIST handwritten digit dataset is used in this case study. It contains grayscale images of handwritten digits from 0 to 9. Each image has a size of 28 × 28 pixels. The dataset includes 60,000 training images and 10,000 testing images.

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training set shape: (60000, 28, 28)
Test set shape: (10000, 28, 28)
Class labels: [0 1 2 3 4 5 6 7 8 9]
```



5. Data Preprocessing

The dataset is loaded using TensorFlow/Keras. Pixel values are normalized from the range 0–255 to 0–1 by dividing them by 255. Normalization helps the neural network learn more efficiently. The training dataset is used to train the model, while the testing dataset is used to evaluate its performance on unseen images.

```
Flattened training data shape: (60000, 784)
Flattened test data shape: (10000, 784)
Pixel value range: [0.0, 1.0]
```

6. Model Development

A neural network is developed using TensorFlow/Keras. The input layer accepts the 28 × 28 pixel images, and the images are flattened into a one-dimensional array. Hidden layers learn important patterns from the images. The output layer contains 10 neurons representing digits 0 to 9. The model uses an appropriate optimizer and loss function for multiclass classification.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

7. Model Training

The neural network is trained using the training dataset over multiple epochs. During training, the model adjusts its weights to reduce prediction errors. Training accuracy and validation accuracy are monitored to understand how effectively the model learns.

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8788 - loss: 0.4034 - val_accuracy: 0.9613 - val
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9451 - loss: 0.1855 - val_accuracy: 0.9737 - val
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9584 - loss: 0.1413 - val_accuracy: 0.9753 - val
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9657 - loss: 0.1157 - val_accuracy: 0.9772 - val
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9683 - loss: 0.1011 - val_accuracy: 0.9795 - val
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9726 - loss: 0.0863 - val_accuracy: 0.9775 - val
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9747 - loss: 0.0811 - val_accuracy: 0.9787 - val
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9764 - loss: 0.0739 - val_accuracy: 0.9783 - val
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9783 - loss: 0.0672 - val_accuracy: 0.9783 - val
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9794 - loss: 0.0616 - val_accuracy: 0.9802 - val
```

8. Model Evaluation

The trained model is evaluated using the testing dataset. Test accuracy measures the proportion of correctly classified images, while test loss measures prediction error. A classification report provides precision, recall, and F1-score for each digit class.

```
Test Loss: 0.0776
Test Accuracy: 0.9774

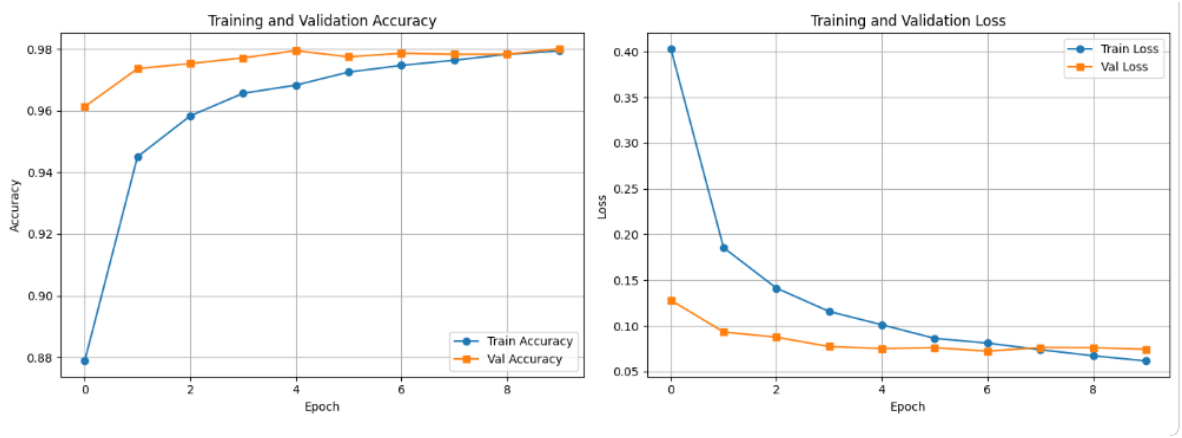
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.99       980
           1       0.99      0.99      0.99      1135
           2       0.97      0.98      0.98      1032
           3       0.97      0.98      0.97      1010
           4       0.98      0.98      0.98       982
           5       0.96      0.99      0.97       892
           6       0.98      0.99      0.98       958
           7       0.99      0.96      0.97      1028
           8       0.96      0.98      0.97       974
           9       0.98      0.95      0.97      1009

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000
```

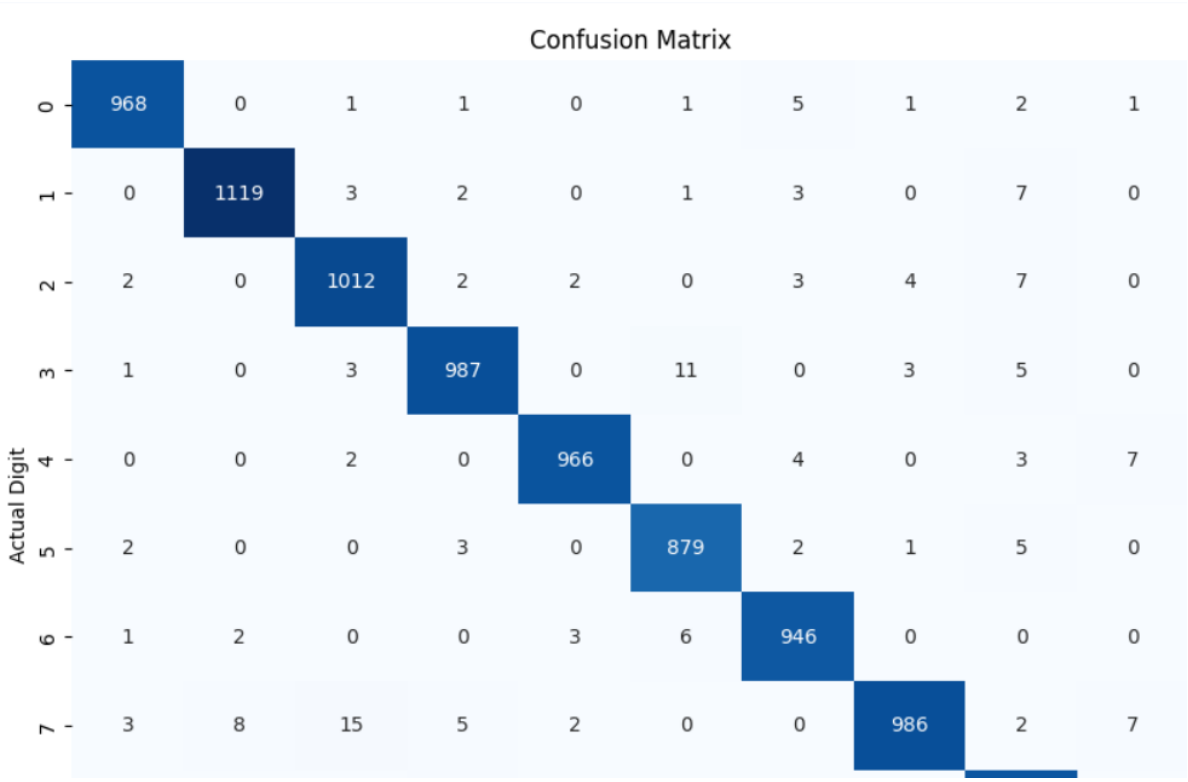
9. Training and Validation Accuracy

The accuracy graph compares training accuracy with validation accuracy across epochs. Increasing accuracy indicates that the model is learning patterns from the dataset. A large difference between the two curves may indicate overfitting.



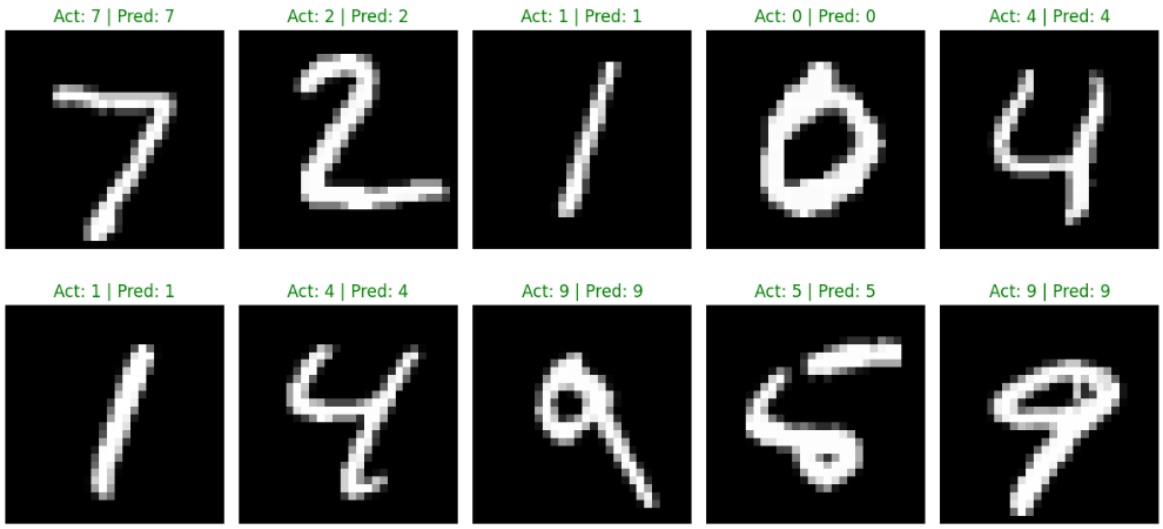
10. Training and Validation Loss

The loss graph shows how the model's prediction error changes during training. A decreasing training loss indicates that the model is learning. If validation loss starts increasing while training loss continues decreasing, the model may be overfitting.



11. Confusion Matrix

A confusion matrix compares the actual digit labels with the predicted labels. Correct predictions appear along the main diagonal, while incorrect predictions appear outside it. This helps identify which handwritten digits the model classifies incorrectly.



12. Actual and Predicted Digit Comparison

The model predicts the digit represented by each test image. Comparing the actual labels with the predicted labels helps demonstrate the model's classification ability and identify examples where predictions are incorrect.

13. Discussion

The experiment demonstrates how deep learning can automatically extract useful features and recognize complex patterns in handwritten digit images. The accuracy and loss graphs help assess learning performance, while the confusion matrix reveals classification errors. Overfitting can be reduced through suitable model design, dropout, or early stopping. The model's performance on unseen test data provides an indication of its generalization ability.

14. Applications

- Handwritten digit recognition
- Optical character recognition
- Document digitization
- Image classification
- Automated data entry systems

15. Conclusion

This case study demonstrates the use of a deep neural network for handwritten digit classification using the MNIST dataset. The model learns features automatically and predicts digits from 0 to 9. Its performance is evaluated using accuracy, loss, a classification report, and a confusion matrix. The experiment illustrates how deep learning helps address feature extraction and complex pattern recognition challenges.

```
Conclusion:
The implemented neural network achieved a high validation accuracy of 98.02%
and a test accuracy of 97.74%.
This highlights the power of deep learning to directly optimize classification
tasks from raw input data, successfully bypassing manual feature engineering.
```

8. APPLICATIONS OF DEEP LEARNING

1. Aim

To study the practical applications of deep learning and implement a deep learning model for image classification using a suitable dataset.

2. Introduction

Deep learning is a branch of machine learning that uses artificial neural networks with multiple layers to learn complex patterns from data. It is widely used in image recognition, speech processing, medical diagnosis, autonomous vehicles, and natural language processing. Deep learning can automatically extract useful features from large datasets and perform classification and prediction tasks.

3. Applications of Deep Learning

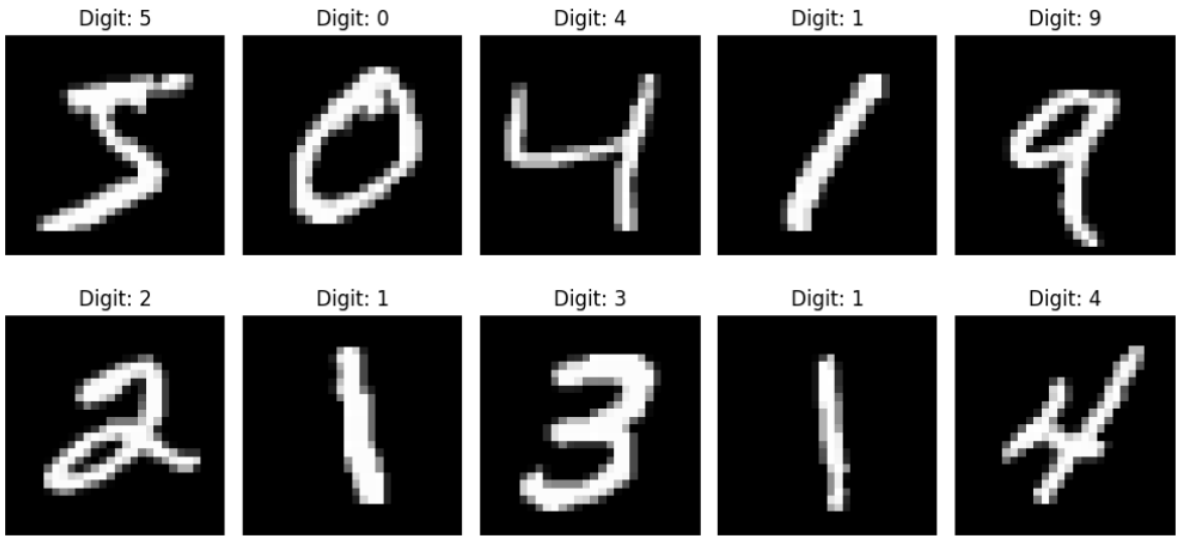
The major applications of deep learning include:

- **Image Recognition:** Identifying objects, animals, and people in images.
- **Healthcare:** Supporting medical image analysis and disease detection.
- **Speech Recognition:** Converting spoken language into text.
- **Natural Language Processing:** Understanding and processing human language.
- **Autonomous Vehicles:** Detecting roads, traffic signs, and obstacles.
- **Security:** Supporting facial recognition and suspicious activity detection.

4. Dataset Description

The MNIST handwritten digit dataset is used to demonstrate a practical application of deep learning. It contains grayscale images of handwritten digits from 0 to 9. Each image has a size of 28 × 28 pixels. The dataset consists of 60,000 training images and 10,000 testing images.

```
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```



5. Data Preprocessing

The dataset is loaded using TensorFlow/Keras. Pixel values are normalized from 0–255 to 0–1 to improve model training. The dataset is divided into training and testing sets. Training data is used to learn patterns, while testing data is used to evaluate the model on unseen images.

```
Normalized training set dimension: (60000, 28, 28, 1)
Normalized testing set dimension: (10000, 28, 28, 1)
Target classes: [0 1 2 3 4 5 6 7 8 9]
```

6. Model Development

A neural network is created using TensorFlow/Keras. The input layer receives the handwritten digit images. The images are flattened, and hidden layers l

7. Model Training

The neural network is trained using the training dataset over multiple epochs. During training, the model adjusts its weights to reduce prediction errors. Training accuracy and validation accuracy are monitored to understand how effectively the model learns patterns from handwritten digit images.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 128)	100,480
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

8. Model Evaluation

The trained model is evaluated using the testing dataset. Test accuracy measures the percentage of correctly classified images, while test loss measures prediction errors. Precision, recall, and F1-score provide additional information about the model's classification performance.

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.8990 - loss: 0.3427 - val_accuracy: 0.9665 - val_
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9534 - loss: 0.1557 - val_accuracy: 0.9737 - val_
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9649 - loss: 0.1125 - val_accuracy: 0.9757 - val_
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9713 - loss: 0.0927 - val_accuracy: 0.9770 - val_
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9747 - loss: 0.0799 - val_accuracy: 0.9792 - val_
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9780 - loss: 0.0683 - val_accuracy: 0.9797 - val_
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9798 - loss: 0.0616 - val_accuracy: 0.9800 - val_
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9829 - loss: 0.0528 - val_accuracy: 0.9805 - val_
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9834 - loss: 0.0514 - val_accuracy: 0.9820 - val_
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9840 - loss: 0.0476 - val_accuracy: 0.9832 - val_
```

9. Training and Validation Accuracy

The accuracy graph compares training accuracy with validation accuracy across epochs. Increasing accuracy indicates that the model is learning useful patterns. A large difference between training and validation accuracy may indicate overfitting.

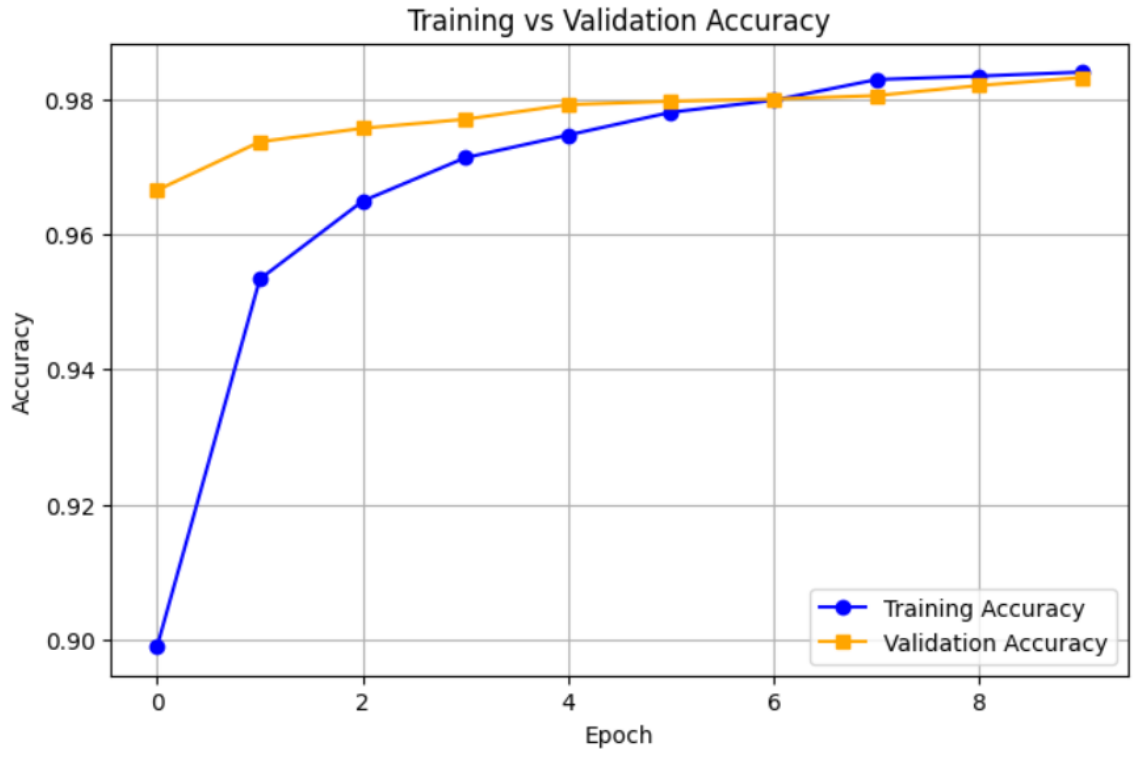
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.98       980
           1       0.99      0.99      0.99      1135
           2       0.98      0.97      0.98      1032
           3       0.96      0.99      0.97      1010
           4       0.98      0.98      0.98       982
           5       0.98      0.97      0.98       892
           6       0.98      0.99      0.98       958
           7       0.98      0.98      0.98      1028
           8       0.98      0.97      0.97       974
           9       0.99      0.96      0.97      1009

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000
```

10. Training and Validation Loss

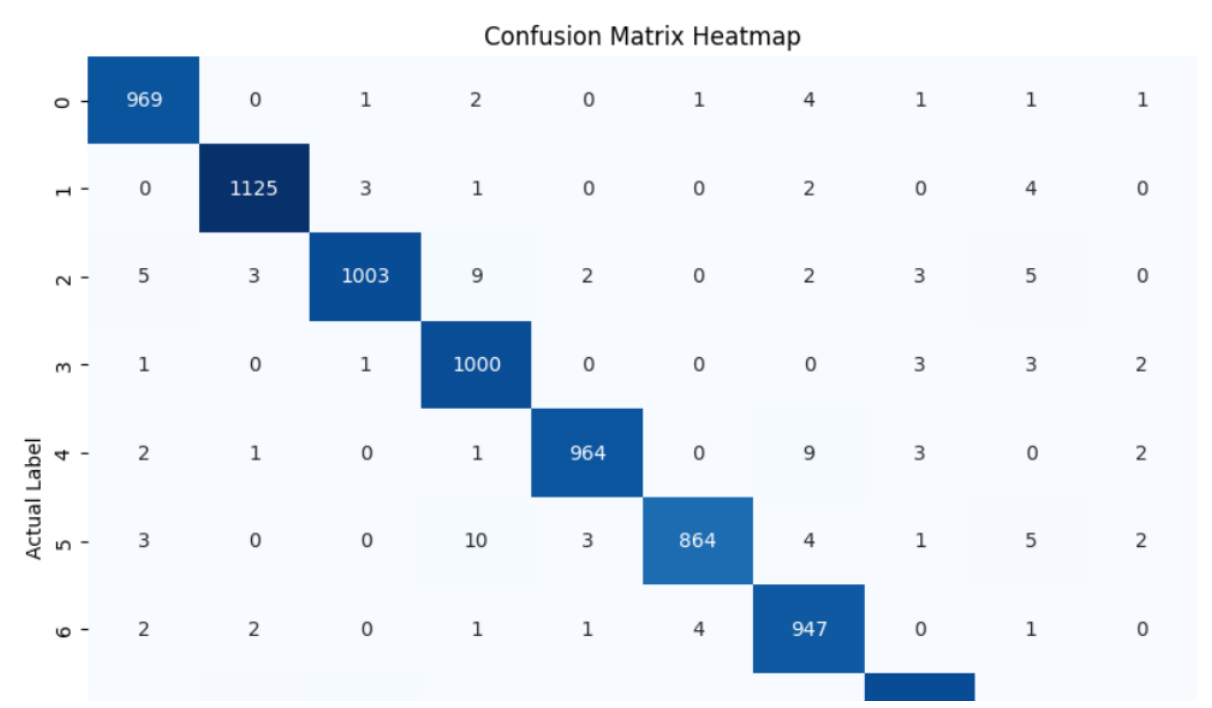
The loss graph shows how prediction errors change during training. Decreasing loss indicates that the model is improving its predictions. If validation loss increases while training loss continues to decrease, the model may be overfitting.





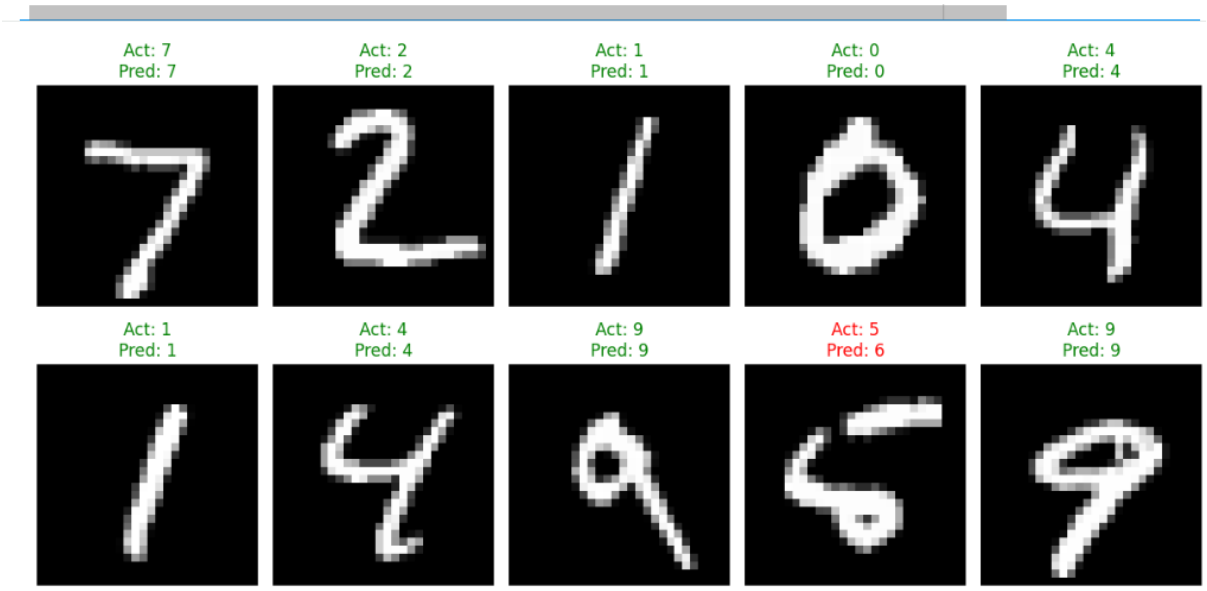
11. Confusion Matrix Analysis

A confusion matrix compares actual digit labels with predicted labels. Values along the main diagonal represent correct classifications, while values outside the diagonal represent incorrect predictions. It helps identify which digits the model finds difficult to distinguish.



12. Actual and Predicted Results

The trained model predicts the digit represented by each test image. Comparing actual labels with predicted labels helps demonstrate the model's classification ability. Incorrect predictions indicate areas where the model could be improved.



13. Discussion

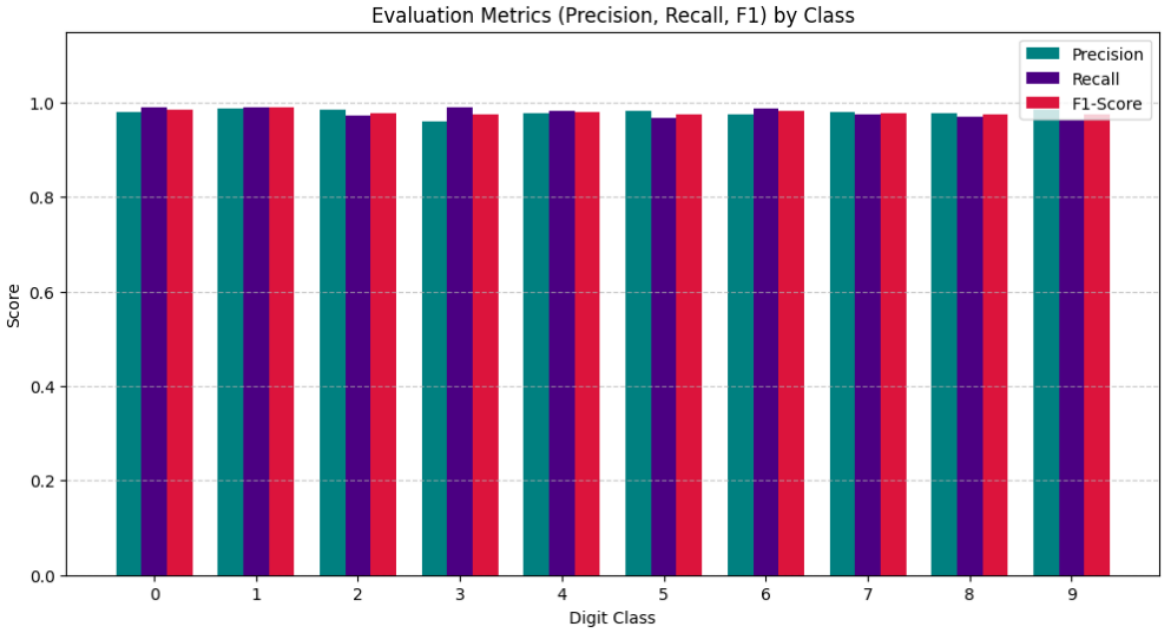
The experiment demonstrates how deep learning can automatically extract useful features and recognize complex patterns in handwritten digit images. The evaluation metrics and graphs help measure performance and identify classification errors. Overfitting can be reduced using techniques such as dropout and early stopping. Testing on unseen images helps assess the model's generalization ability.

14. Applications

- Handwritten digit recognition
- Optical character recognition
- Document processing
- Medical image analysis
- Automated data entry systems
- Image classification

15. Conclusion

This case study demonstrates the practical application of deep learning in handwritten digit classification using the MNIST dataset. The neural network learns features automatically and predicts digits from 0 to 9. Its performance is evaluated using accuracy, loss, classification metrics, and a confusion matrix. The experiment highlights the importance of deep learning in modern intelligent applications.



9. STOCHASTIC GRADIENT DESCENT

1. Aim

To implement linear regression using Stochastic Gradient Descent (SGD), compare SGD with Batch Gradient Descent in terms of efficiency and convergence, and analyze the effect of different learning rates on model performance.

2. Introduction

Stochastic Gradient Descent (SGD) is an optimization algorithm used to minimize the loss function of a machine learning model. It updates model parameters using one training sample at a time. This makes SGD useful for large datasets because it can update parameters without processing the entire dataset for every iteration.

Batch Gradient Descent calculates the gradient using the complete training dataset before updating the parameters. SGD updates parameters more frequently, which can reduce computation per update. However, its updates may fluctuate because each update depends on an individual sample.

The learning rate controls the size of each parameter update. A suitable learning rate helps the model converge efficiently, while a very high or low learning rate can affect training performance.

3. Dataset Description

A suitable regression dataset is used to predict a continuous target value from input features. The dataset is divided into training and testing sets. The training set is used to learn the model parameters, while the testing set is used to evaluate predictions on unseen data.

First 5 rows of the dataset:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

4. Data Preprocessing

The dataset is checked for missing values and prepared for model training. Input features are separated from the target variable. The data is divided into training and testing sets using a fixed random state to ensure reproducible results. Feature scaling is applied when required so that features with different scales do not negatively affect gradient-based optimization.

```
Training set shape: (16512, 8)
Testing set shape: (4128, 8)
First row of scaled training data (X_train[0]):
[-0.326196    0.34849025 -0.17491646 -0.20836543  0.76827628  0.05137609
 -1.3728112   1.27258656]
```

5. Implementation of Batch Gradient Descent and SGD

Two linear regression optimization methods are implemented and compared.

Batch Gradient Descent: The gradient is calculated using all training samples before each parameter update. This provides a more stable direction for reducing the loss but can require more computation per update for large datasets.

Stochastic Gradient Descent: The gradient is calculated using one randomly selected training sample at a time. Parameters are updated frequently, making the method suitable for large-scale learning.

Both methods aim to minimize the Mean Squared Error (MSE) loss function.

[
MSE=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2
]

where (y_i) is the actual value, (\hat{y}_i) is the predicted value, and (n) is the number of samples.

```
Training Custom Batch Gradient Descent...
BGD complete. Final iteration loss: 0.29148. Time taken: 0.0886 seconds.

Training Scikit-Learn SGDRegressor...
SGD complete. Final epoch loss: 0.25933. Time taken: 2.1149 seconds.
```

6. Learning Rate Experiment

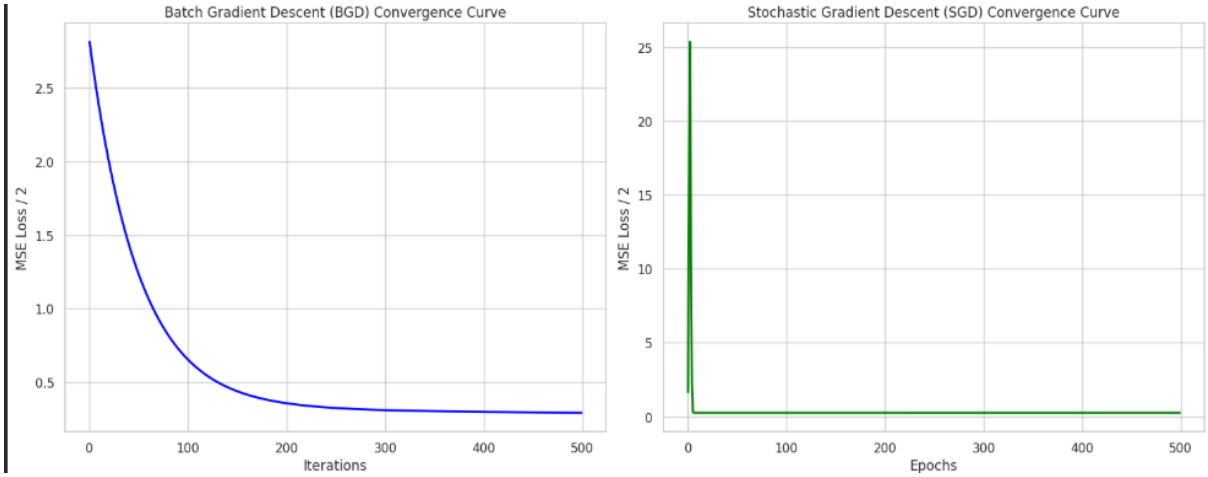
The learning rate determines how much the model parameters change during each update. Different learning rates, such as 0.001, 0.01, and 0.1, are tested to study their effects.

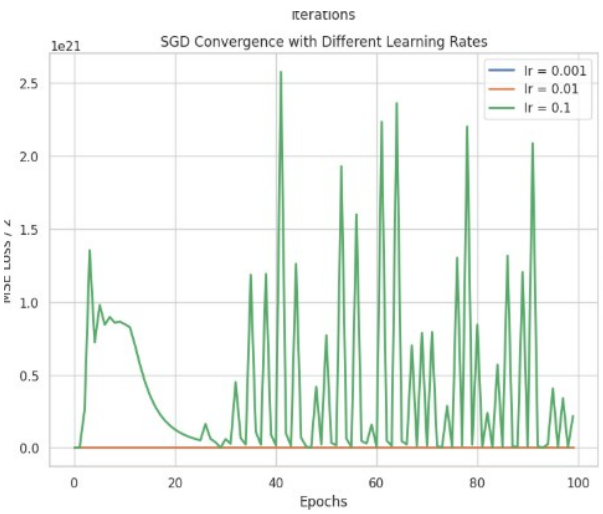
- **Low learning rate:** Updates are small and training may take longer to converge.
- **Suitable learning rate:** The loss decreases efficiently and training progresses steadily.
- **High learning rate:** Updates may become unstable, causing the loss to fluctuate or increase.

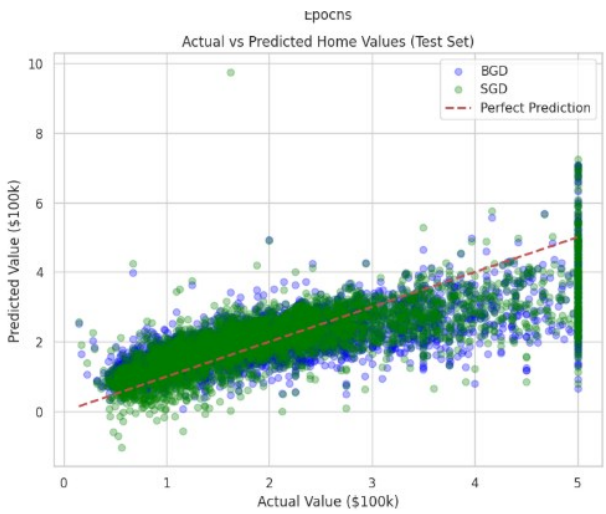
The best learning rate depends on the dataset, feature scale, and optimization method.

```
Testing learning rate: 0.001...
Testing learning rate: 0.01...
Testing learning rate: 0.1...
```

	Final Train Loss	Test MSE	Test RMSE
0.001	2.590035e-01	5.547962e-01	7.448464e-01
0.010	2.594827e-01	5.490426e-01	7.409741e-01
0.100	2.174727e+20	1.085438e+19	3.294598e+09







7. Model Training and Convergence

Both optimization methods are trained on the same training dataset. The loss values are recorded during training to observe how quickly each method approaches a minimum. A loss-versus-iteration graph is used to compare their convergence behavior.

Batch Gradient Descent generally produces smoother loss changes because it uses the complete dataset for each update. SGD may show more fluctuations but can make frequent updates with lower computation per update.

8. Model Evaluation

The trained models are evaluated using the test dataset. Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) are used to measure prediction errors.

[
MAE=\frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i|
]

[
RMSE=\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}
]

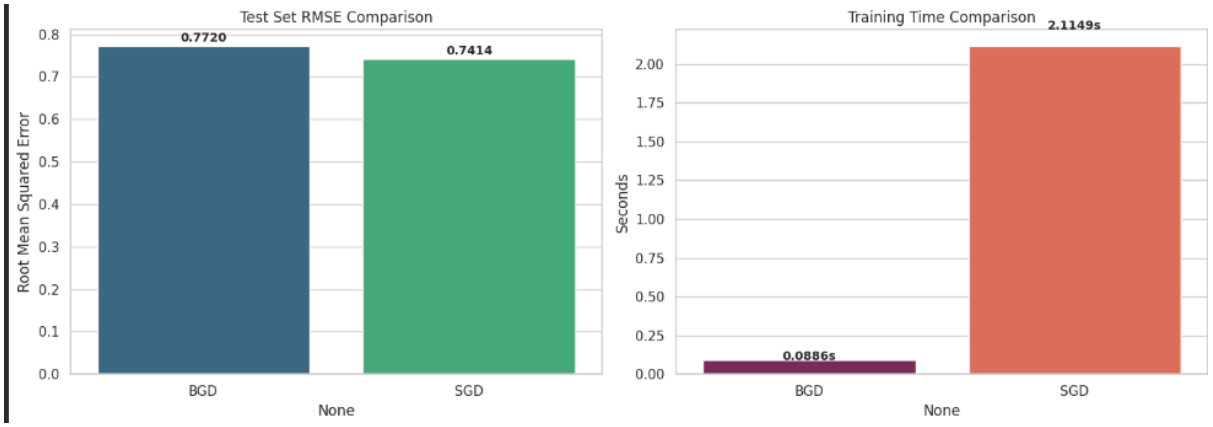
Lower MAE and RMSE values indicate smaller prediction errors. The actual metrics obtained from the executed program are recorded for comparison.

	MAE	MSE	RMSE	R2 Score	Training Time (s)
BGD	0.565250	0.596056	0.772047	0.545138	0.088634
SGD	0.534411	0.549633	0.741373	0.580564	2.114875

```
/tmp/ipykernel_1230/3537381165.py:225: FutureWarning:
```

9. Efficiency Comparison

The training time and evaluation metrics of Batch Gradient Descent and SGD are compared. Batch Gradient Descent performs each update using all training samples, whereas SGD performs updates using individual samples. SGD can be advantageous for large datasets, although its total training time and final accuracy depend on implementation, learning rate, and stopping criteria.



10. Results and Discussion

The experiment demonstrates how gradient-based optimization can be used to train a linear regression model. Batch Gradient Descent provides updates based on the complete dataset, while SGD updates the parameters using individual samples. The learning rate significantly affects convergence speed and stability. The final comparison should be based on the actual training time, loss curves, and test metrics produced by the program.

11. Conclusion

Stochastic Gradient Descent was implemented for linear regression and compared with Batch Gradient Descent. Different learning rates were tested to understand their influence on convergence and model performance. The experiment shows that the choice of optimization method and learning rate affects training efficiency and prediction quality. SGD is particularly useful when frequent parameter updates are beneficial for large datasets, while the best method should be selected based on measured performance.

10. APPLICATIONS OF DEEP LEARNING

1. Aim

To study and implement an application of deep learning using a neural network for handwritten digit recognition and evaluate its performance using the MNIST dataset.

2. Introduction

Deep learning is a branch of machine learning that uses artificial neural networks with multiple layers to learn patterns from data. It can automatically extract useful features from images, text, audio, and other data.

Deep learning is widely used in image recognition, healthcare, speech recognition, natural language processing, autonomous vehicles, and security systems. In this case study, a neural network is used to recognize handwritten digits from images.

3. Applications of Deep Learning

The major applications of deep learning include:

1. **Image Recognition:** Identifying objects, faces, and handwritten characters in images.
2. **Healthcare:** Supporting medical image analysis and disease detection.
3. **Speech Recognition:** Converting spoken language into text.
4. **Natural Language Processing:** Supporting translation, chatbots, and text classification.
5. **Autonomous Vehicles:** Detecting roads, traffic signs, pedestrians, and other vehicles.
6. **Security Systems:** Supporting face recognition and suspicious activity detection.

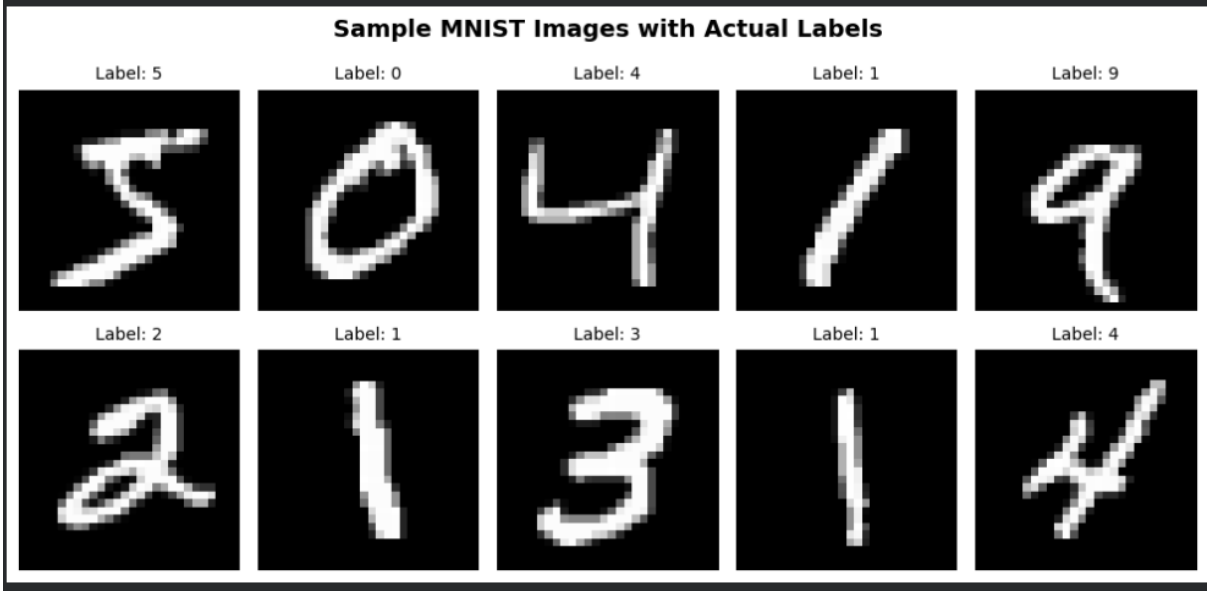
4. Dataset Description

The MNIST dataset is used for handwritten digit recognition. It contains 70,000 grayscale images of handwritten digits from 0 to 9.

- Training images: 60,000
- Testing images: 10,000
- Image size: 28 × 28 pixels
- Number of classes: 10
- Pixel values: 0 to 255

Each image represents one handwritten digit, and the model learns to predict the correct digit.

```
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Raw Training Images Shape: (60000, 28, 28)
Raw Training Labels Shape: (60000,)
Raw Testing Images Shape: (10000, 28, 28)
Raw Testing Labels Shape: (10000,)
Number of Training Samples: 60000
Number of Testing Samples: 10000
Image Dimensions: 28x28
Number of Classes: 10
```



5. Data Preprocessing

The image data is preprocessed before training the neural network. Pixel values are normalized by dividing them by 255 so that they fall between 0 and 1. This helps improve the stability of training.

The images are reshaped into one-dimensional vectors when using a fully connected neural network. The original digit labels, ranging from 0 to 9, are used as the target classes.

```
Preprocessed Training Images Shape: (60000, 28, 28)
Preprocessed Testing Images Shape: (10000, 28, 28)
Preprocessed Train Pixels Min/Max: 0.0 to 1.0
```

6. Model Development

A feedforward neural network is developed using TensorFlow and Keras. The model contains an input layer, hidden layers, and an output layer.

- **Input layer:** Receives the 28 × 28 pixel image.
- **Hidden layers:** Learn patterns and features from the image.
- **Output layer:** Contains 10 neurons, representing digits from 0 to 9.

- **Activation function:** ReLU is used in the hidden layers, and Softmax is used in the output layer to produce class probabilities.

The model is compiled using the Adam optimizer and sparse categorical cross-entropy loss.

Model: "sequential"

Layer (type)	Output Shape	Param #
Flatten_Layer (Flatten)	(None, 784)	0
Hidden_Layer_1 (Dense)	(None, 128)	100,480
Dropout_Layer (Dropout)	(None, 128)	0
Hidden_Layer_2 (Dense)	(None, 64)	8,256
Output_Layer (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

7. Model Training

The neural network is trained using the training dataset. During training, the model compares its predictions with the actual labels and updates its weights to reduce the loss.

Training and validation accuracy and loss are recorded for each epoch. These values help evaluate whether the model is learning effectively.

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.8981 - loss: 0.3408 - val_accuracy: 0.9670 - v
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.9534 - loss: 0.1537 - val_accuracy: 0.9725 - va
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9656 - loss: 0.1141 - val_accuracy: 0.9748 - va
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9711 - loss: 0.0941 - val_accuracy: 0.9758 - va
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - accuracy: 0.9745 - loss: 0.0803 - val_accuracy: 0.9777 - va
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9773 - loss: 0.0695 - val_accuracy: 0.9785 - va
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9794 - loss: 0.0637 - val_accuracy: 0.9792 - va
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9810 - loss: 0.0589 - val_accuracy: 0.9802 - va
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9830 - loss: 0.0512 - val_accuracy: 0.9797 - va
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9844 - loss: 0.0467 - val_accuracy: 0.9787 - va
```

8. Model Evaluation

After training, the model is evaluated using the test dataset, which contains images not used for training. Test accuracy and test loss are calculated to measure the model's performance on unseen images.

A classification report can also be generated to calculate precision, recall, and F1-score for each digit class. A confusion matrix helps identify which digits are correctly classified and which digits are confused with other digits.

```
Test Loss: 0.0763
Test Accuracy: 0.9770
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       980
           1       0.99      0.99      0.99      1135
           2       0.95      0.99      0.97      1032
           3       0.98      0.97      0.97      1010
           4       0.98      0.98      0.98       982
           5       0.97      0.98      0.98       892
           6       0.97      0.99      0.98       958
           7       0.98      0.96      0.97      1028
           8       0.98      0.97      0.98       974
           9       0.99      0.96      0.97      1009

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000
```

9. Training and Validation Accuracy

A graph is plotted to compare training accuracy and validation accuracy across epochs.

Training accuracy shows how well the model performs on training data. Validation accuracy shows how well it performs on separate validation data. If both values increase and remain close, the model is generally learning effectively. A large difference between them may indicate overfitting.



10. Training and Validation Loss

A loss graph is plotted to observe changes in training and validation loss during training. Decreasing loss indicates that the model is improving its predictions.

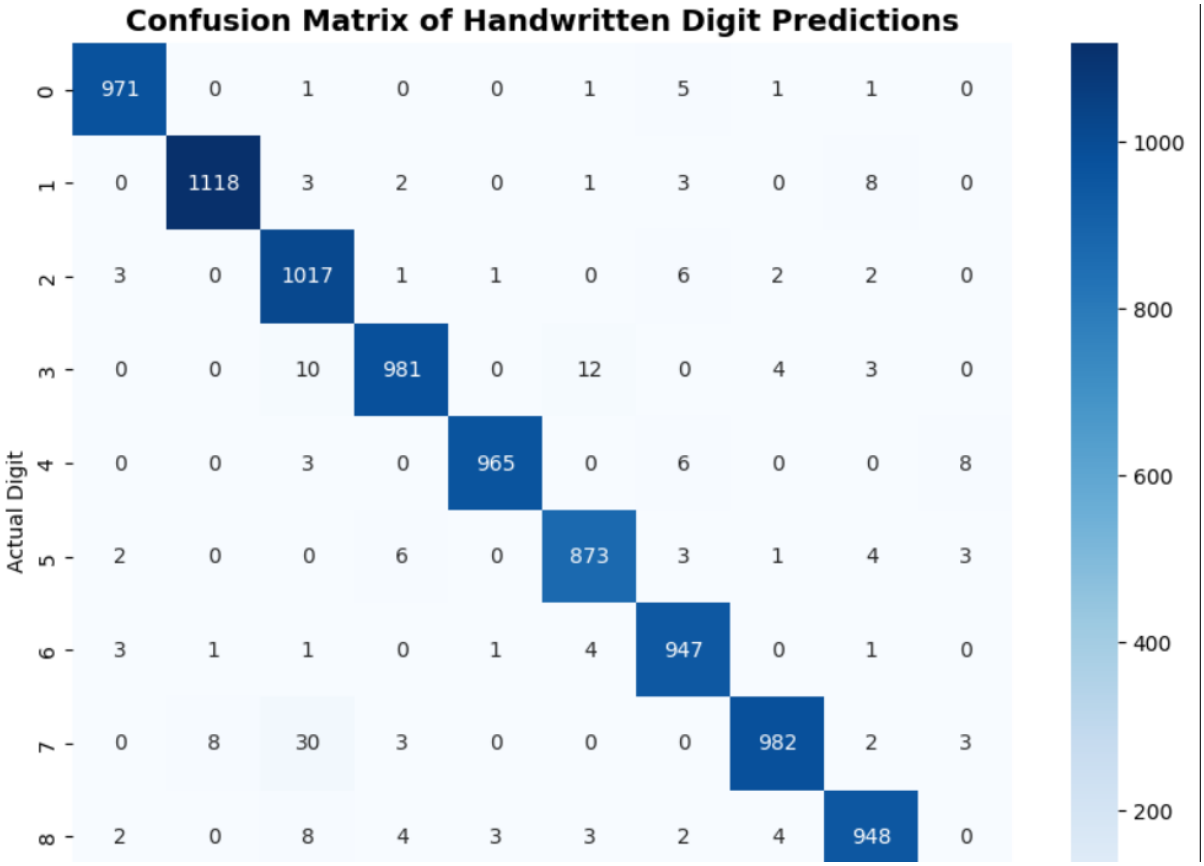
If training loss decreases while validation loss increases, the model may be overfitting. The graph helps determine whether the model is learning effectively and generalizing to unseen data.



11. Confusion Matrix

A confusion matrix is generated to compare the actual digit labels with the predicted digit labels. Correct predictions appear along the main diagonal, while incorrect predictions appear outside it.

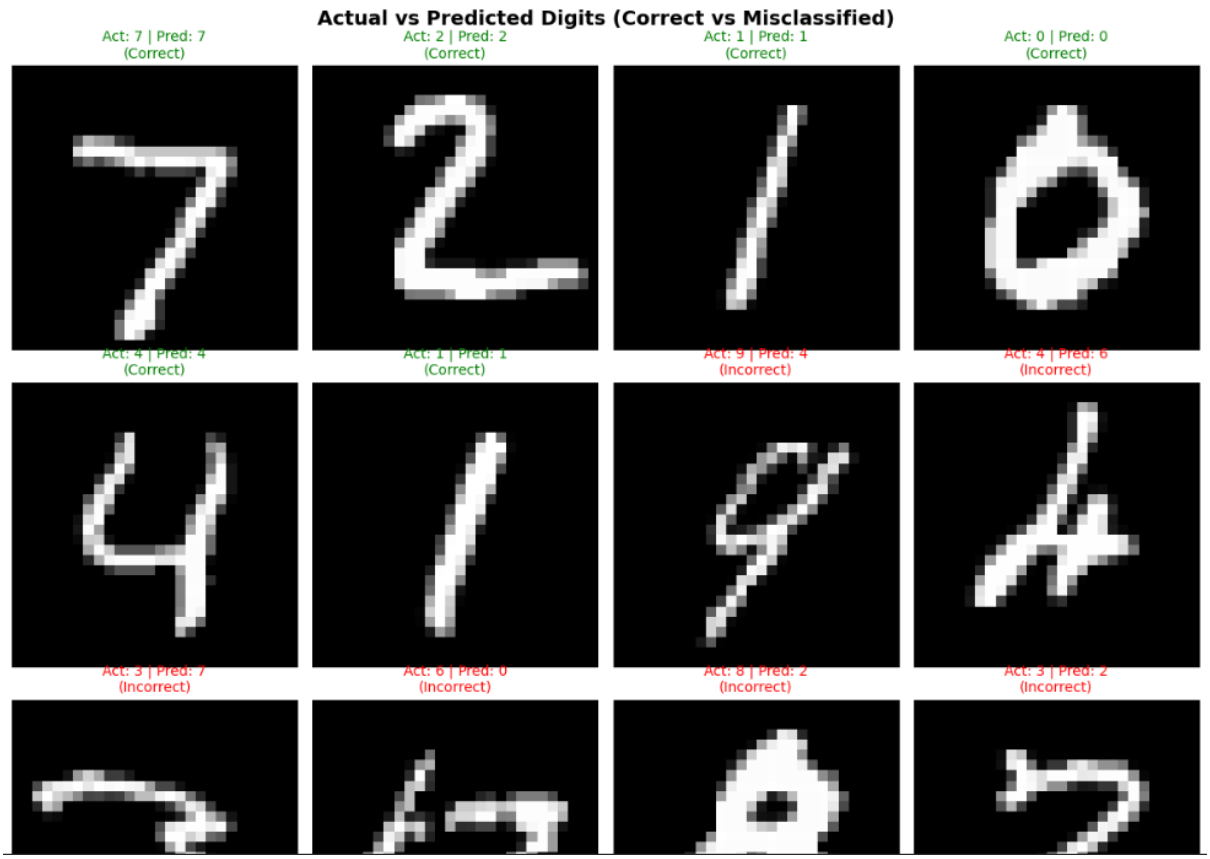
This matrix helps identify digits that the model recognizes accurately and digits that it frequently misclassifies.



12. Actual and Predicted Digit Comparison

Sample test images are displayed with their actual labels and predicted labels. This provides a visual demonstration of the model's ability to recognize handwritten digits.

Correct predictions demonstrate successful recognition, while incorrect predictions help identify cases where the model needs improvement.



13. Results and Discussion

The deep learning model learns patterns from handwritten digit images and uses them to classify digits into ten categories. Its performance is evaluated using test accuracy, loss, precision, recall, F1-score, and the confusion matrix.

The accuracy and loss graphs help explain the model's learning behavior. The actual and predicted image comparison provides a practical demonstration of the application of deep learning in image recognition.

The final performance values should be taken from the executed program.

14. Advantages and Limitations

Advantages:

- Automatically learns useful features from images.
- Can classify many handwritten digit patterns.
- Supports practical image recognition applications.
- Can be improved by using more data and suitable model architectures.

Limitations:

- Requires sufficient training data and computational resources.
- May misclassify unclear or unusual handwriting.
- Performance depends on model design and training settings.
- A model trained on MNIST may not perform equally well on real-world images that differ from the dataset.

15. Conclusion

This case study demonstrates an application of deep learning through handwritten digit recognition using the MNIST dataset. A neural network is trained and evaluated using test data, accuracy and loss graphs, a confusion matrix, and sample predictions. The experiment illustrates how deep learning can automatically learn image features and perform classification. The same general approach can be adapted to other applications, including medical image analysis, object recognition, and document processing.

